

Welcome to Yantrika

Induction Session 2026

Student-Run Robotics and Coding Club

Research Scholar Association, IACS

August 2026

YANTRIKA

Robotics and Coding Club of IACS

Imagine. Build. Experiment. Repeat.

Why Yantrika?

- Robotics brings together **physics, mathematics, electronics, computation, and engineering**.
- It provides an opportunity to move from:

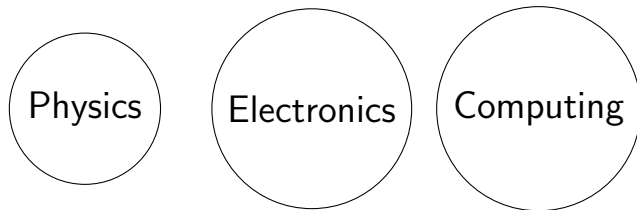
Learning concepts $\longrightarrow$ **Building systems**

- Yantrika aims to create a culture of:
 - Hands-on experimentation
 - Collaborative problem solving
 - Open-ended projects
 - Research-oriented engineering

**We don't just learn technology.
We learn by building it.**

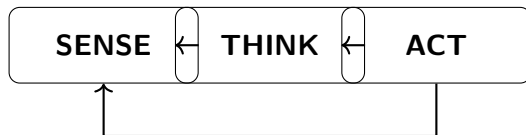
What is Robotics?

Robotics is the intersection of



- A robot interacts with the **physical world**.
- It can **sense**, **process information**, and **act**.
- Autonomy can range from remote control to fully autonomous operation.

The Robotics Loop



SENSE

LiDAR
Camera
IMU
Encoders

THINK

Microcontrollers
Raspberry Pi
Algorithms
AI

ACT

Motors
Servos
Wheels
Propellers

From Science to Robotics

Discipline	Concept	Robotic Application
Physics	Mechanics	Robot motion
Physics	Optics	Computer vision
Mathematics	Linear algebra	Robot localization
Mathematics	Probability	Sensor fusion
Electronics	Circuits	Sensors & actuators
Computer Science	Algorithms	Navigation
AI	Machine Learning	Perception

Robotics turns individual concepts into complete systems.

What Can We Explore?

Autonomous Robotics

- Mobile robots
- SLAM
- Path planning
- ROS 2
- Sensor fusion

Drones

- Flight control
- Navigation
- Computer vision
- Obstacle avoidance

Embedded Systems

- Microcontrollers
- Sensors
- PCB design
- Motor control

Coding & Software

- Python, C/C++ and embedded programming
- Algorithms and computational thinking
- Computer vision and AI
- Simulation and data visualization
- GUI and robotics software development

Where Do You Fit in Yantrika?

Build

- Mechanical Design
- Electronics
- Embedded Systems
- PCB Design

Program

- Python
- C/C++
- Algorithms
- ROS 2

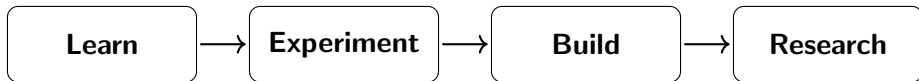
Make It Autonomous

- Computer Vision
- AI / Machine Learning
- SLAM
- Control Systems

Connect It

- RF
- LoRa
- IoT
- Wireless Networks

Your Journey with Yantrika



- You don't need to be an expert.
- You don't need to know robotics already.
- You don't need to know every programming language.
- **Curiosity and willingness to learn are enough to start.**

Key Components of Robotics

Sensors

- Cameras
- LiDAR
- Ultrasonic sensors
- IMU, encoders

Actuators

- DC motors
- Servo motors
- Stepper motors
- Robotic arms / wheels

Controller

- Microcontrollers
- Single-board computers
- Processes sensor data
- Executes control algorithms

Software

- Embedded programming
- Computer vision
- Path planning
- AI & machine learning

The Controller

What does a Controller do?

- Acts as the “brain” of a robotic system.
- Receives information from sensors.
- Processes the input according to a program.
- Sends commands to actuators.

Input → Processing → Output

Examples

- Microcontrollers
- Single-board computers
- PLCs
- Embedded processors

The controller turns decisions into actions.

What is a Microcontroller?

Microcontroller (MCU)

- A small computer integrated into a single chip.
- Contains a processor, memory, and peripherals.
- Designed to control electronic and embedded systems.
- Usually programmed to perform a specific task.

Inside an MCU

- CPU / Processing Core
- Flash Memory
- RAM
- GPIO Pins
- Timers
- ADC / PWM
- Communication interfaces
- USB / Serial interfaces

Sensors → MCU → Actuators

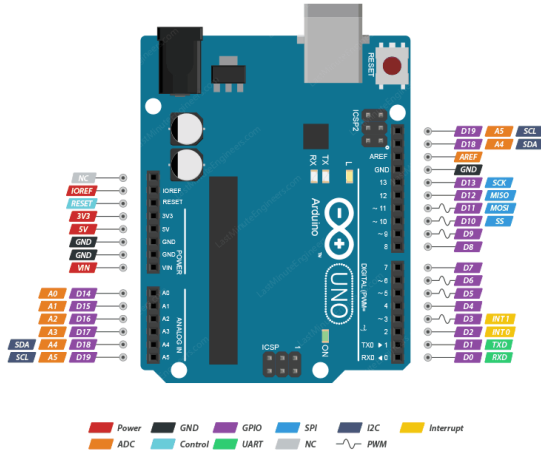
Arduino Uno

- Beginner-friendly microcontroller board.
- Based on the **ATmega328P**.
- 8-bit AVR microcontroller.
- Clock frequency: **16 MHz**.
- 32 KB Flash memory.
- 2 KB SRAM.
- 1 KB EEPROM.
- 14 digital I/O pins.
- 6 analog input pins.

Why is it useful in robotics?

- Sensors and actuators
- Motor and servo control
- Embedded programming
- Rapid prototyping

Arduino Uno



Arduino UNO R3 Pinout

Arduino Uno Pin Breakdown

Digital Pins

Pin	Function
D0	RX
D1	TX
D2	Digital I/O, INT0
D3	Digital I/O, PWM, INT1
D4	Digital I/O
D5	Digital I/O, PWM
D6	Digital I/O, PWM
D7	Digital I/O
D8	Digital I/O
D9	Digital I/O, PWM
D10	Digital I/O, PWM, SPI SS
D11	Digital I/O, PWM, SPI MOSI
D12	Digital I/O, SPI MISO
D13	Digital I/O, SPI SCK, LED

PWM Pins

D3, D5, D6, D9, D10, D11

Analog Pins

Pin	Function
A0	Analog Input 0
A1	Analog Input 1
A2	Analog Input 2
A3	Analog Input 3
A4	Analog Input, I ² C SDA
A5	Analog Input, I ² C SCL

Important

- A0–A5 can also be used as digital pins.
- D2 and D3 support external interrupts.

D2 = INT0 D3 = INT1

Arduino Uno: Communication & Power

UART / Serial

- D0: RX
- D1: TX

I²C

- A4: SDA
- A5: SCL

SPI

- D10: SS
- D11: MOSI
- D12: MISO
- D13: SCK

Power Pins

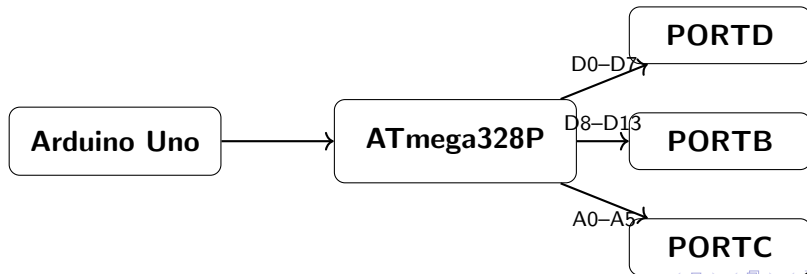
- VIN
- 5V
- 3.3V
- GND
- RESET
- IOREF

UART → D0, D1
I²C → A4, A5
SPI → D10 – D13

Arduino Uno: Inside the ATmega328P

Arduino Pin → ATmega328P Port Mapping

Arduino Pins	MCU Port	Port Bits
D0–D7	PORTD	PD0–PD7
D8–D13	PORTB	PB0–PB5
A0–A5	PORTC	PC0–PC5



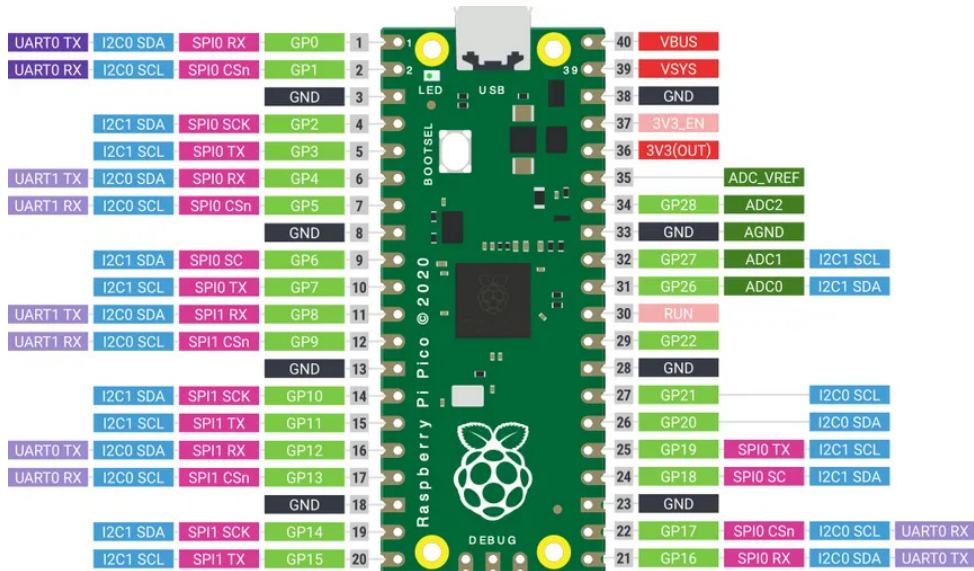
Raspberry Pi Pico

- Based on the **RP2040**.
- Dual-core ARM Cortex-M0+.
- Up to **133 MHz**.
- 26 multifunction GPIO pins.
- C/C++ or MicroPython.

Useful for Robotics

- Sensors
- Motors and servos
- PWM control
- Communication
- Real-time control
- Embedded projects

Raspberry Pi Pico



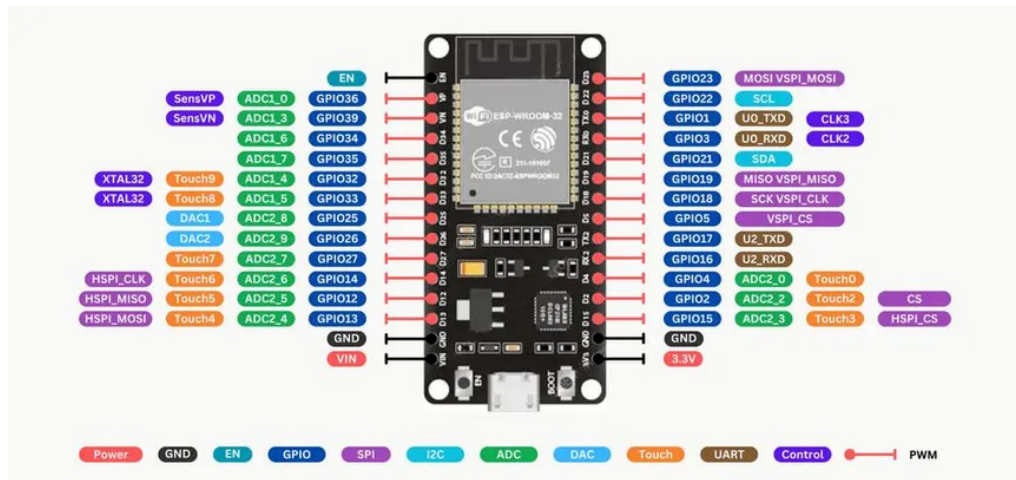
ESP32

- Powerful microcontroller family by Espressif.
- Multiple ESP32 variants are available.
- Built-in **Wi-Fi** and **Bluetooth**.
- Multiple GPIO, ADC, PWM, UART, SPI and I²C interfaces.

Why ESP32 for Robotics?

- Wireless robot control
- IoT applications
- Sensor networks
- Bluetooth-controlled robots
- Web-based control
- Robot-to-robot communication

ESP32



How Do Robotic Systems Communicate?

A robot is not one computer.

It is a collection of devices that must exchange information.



Distance

Bandwidth

Reliability

Power

Number of Devices

UART — Simple Serial Communication

UART

- Asynchronous serial communication.
- Usually point-to-point.
- TX and RX lines.



Typical Robotics Uses

- GPS modules
- Bluetooth modules
- Serial debugging
- MCU-to-MCU communication
- Flight controllers

Further Learning

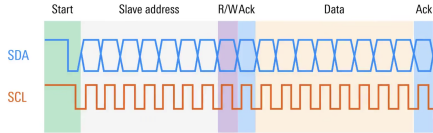
UART Explained in Detail

<https://youtu.be/sTHckUyxwp8>

I²C — Shared Sensor Bus

Two-wire bus

- SDA — Data
- SCL — Clock
- Multiple devices share the bus.
- Devices use addresses.



Typical Robotics Uses

- IMU
- Accelerometer
- Gyroscope
- Magnetometer
- OLED
- Temperature sensors

Further Learning

I2C Explained in Detail

<https://youtu.be/CAvawEcxoPU?si=biwvrDOG8U0i1eux>

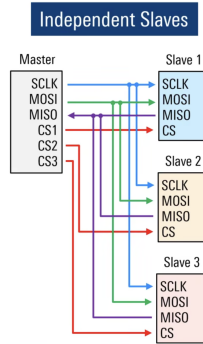
SPI — Bus Connections

SPI Signals

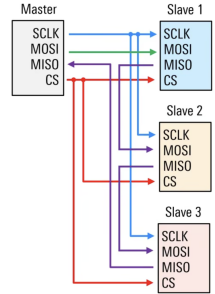
- **SCLK** — Serial Clock
- **MOSI** — Master Out, Slave In
- **MISO** — Master In, Slave Out
- **CS** — Chip Select

Key Idea

- Clock and data lines are shared.
- Each slave has its own **CS** line.
- Only the selected slave communicates.

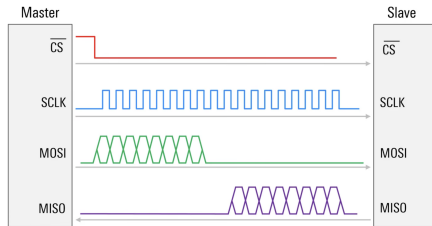


Cooperative Slaves / Daisy Chain



Multiple SPI devices sharing the same bus

SPI — How Data Is Transferred



Basic sequence

CS LOW → SCLK pulses → MOSI/MISO transfer → CS HIGH

Further Learning

SPI Explained in Detail

<https://youtu.be/OnVNwozXsIc?si=6yBIs7Di6ewPJB5Q>

UART vs I²C vs SPI

Feature	UART	I ² C	SPI
Clock	No	Yes	Yes
Lines	TX/RX	SDA/SCL	SCLK/MOSI/MISO/CS
Multiple Devices	Limited	Yes	Yes
Speed	Medium	Medium	High
Typical Use	GPS / Debug	Sensors	Fast peripherals

UART = Simple

I²C = Shared

SPI = Fast

Different parts of a robot use different protocols.

Board Level

UART
I²C
SPI

Robot Level

CAN
RS-485

Wireless

Wi-Fi
Bluetooth
LoRa

Software

USB
Ethernet
ROS 2 / DDS

Protocol choice depends on the application.

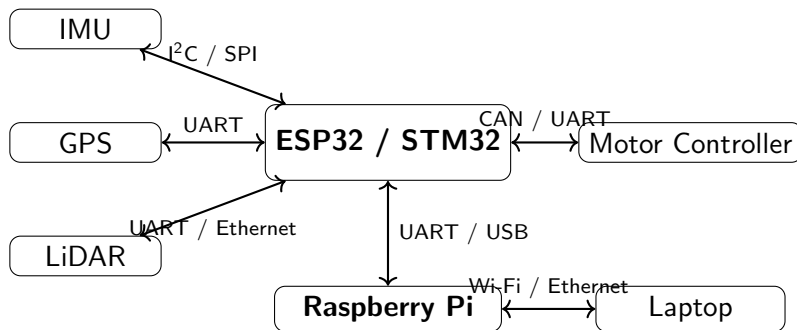
So Which Protocol Do We Choose?

There is no “best” protocol.

Protocol	Range	Speed	Nodes	Typical Use
UART	Short	Low–Med	2	GPS / Debug
I ² C	Short	Low–Med	Many	Sensors
SPI	Short	High	Several	Fast peripherals
CAN	Medium	Medium	Many	Motor control
RS-485	Long	Medium	Many	Industrial
USB	Short	High	Multiple	PC
Wi-Fi	Medium	High	Many	Robot network
BLE	Short	Low	Few	Phone control
LoRa	Very Long	Low	Many	Telemetry
Ethernet	Long	Very High	Many	ROS 2 / PC

Ask: How far? How much data? How many devices? How reliable?

Putting It Together: A Robotic System



One robot can use many protocols at different levels.

Interfacing with Arduino IDE

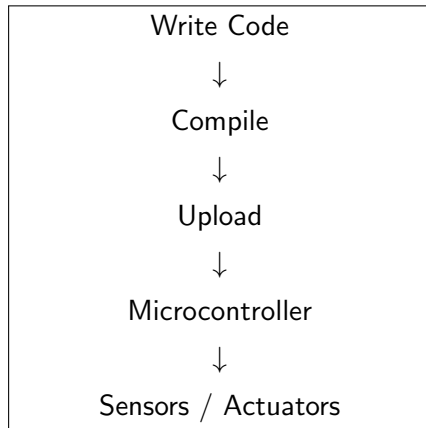
What is Arduino IDE?

- Integrated Development Environment used to program Arduino boards.
- Provides an editor, compiler, uploader and Serial Monitor.
- Programs are written as Arduino **Sketches**.
- Based primarily on **C/C++**.

Main Tasks

- Write and edit code
- Verify and compile
- Upload firmware
- Debug using Serial Monitor

Development Workflow



Arduino Sketch



Setting Up Arduino IDE & SimulIDE

1. Arduino IDE

① Install Arduino IDE

- Install Arduino IDE.
- Connect the Arduino using USB.

② Select the Board

- Tools → Board
- Select the appropriate Arduino board.

③ Select the Port

- Tools → Port
- Select the Arduino serial port.

2. SimulIDE

① Install SimulIDE

- Download and install SimulIDE.
- Launch the simulator.

② Create a Circuit

- Add an Arduino board.
- Add LEDs, resistors, sensors, etc.
- Connect the components.

③ Prepare the Simulation

- Select the required Arduino model.
- Build the required circuit.

Two environments, two purposes

Arduino IDE = Program

SimulIDE = Simulate

Arduino Programming: Program Structure

Every Arduino sketch has two main functions:

setup()

Runs **once** when the Arduino is powered on or reset.

- Configure pins
- Initialize sensors and modules
- Start Serial communication

loop()

Runs **continuously** after setup().

- Read inputs
- Process information
- Control outputs

Program Execution

Power ON / RESET



setup()



loop()



Important:

setup() executes once, while loop() keeps executing repeatedly.

Arduino Programming: A Simple Example

LED Blink Program

```
int LED = 9;

void setup()
{
    pinMode(LED, OUTPUT);
}

void loop()
{
    digitalWrite(LED, HIGH);
    delay(1000);

    digitalWrite(LED, LOW);
    delay(1000);
}
```

Understanding the Code

- `int LED = 13;`
Stores the LED pin number.
- `pinMode()`
Configures the pin as an output.
- `digitalWrite()`
Sets the output to HIGH or LOW.
- `delay(1000)`
Pauses execution for 1000 ms.

Execution

ON → Wait 1 s → OFF → Wait 1 s → Repeat

LED Fading using PWM

Part 1 — Setup and Fade In

```
int LED = 9;

void setup()
{
  pinMode(LED, OUTPUT);
}

void loop()
{
  // Fade in
  for (int brightness = 0;
       brightness <= 255;
       brightness++)
  {
    analogWrite(LED, brightness);
    delay(10);
  }
}
```

What happens here?

- LED is connected to **PWM pin D9**.
- `pinMode()` configures D9 as an output.
- The for loop increases the brightness value.
- `analogWrite()` generates the PWM signal.

0 → 255

LED Fading — Part 2

Part 2 — Fade Out

```
// Fade out
for (int brightness = 255;
    brightness >= 0;
    brightness--)
{
    analogWrite(LED, brightness);
    delay(10);
}
```

The complete cycle is:

Fade Out

- Start at maximum brightness.
- The brightness value decreases from **255 to 0**.
- The LED gradually becomes dimmer.

255 → 0

BRIGHT → OFF

Then `loop()` repeats.

Learning Through Workshops

Tutorial-Based Workshops

Yantrika follows a structured, course-like approach to learning.

- Open to all students
- No prior experience required
- Step-by-step guidance from fundamentals
- Hands-on learning through practical activities

Learning Resources

All resources remain accessible even after the sessions.

- Notes, materials and simulations uploaded to GitHub
- Session schedules and updates on the official website
- Self-paced learning outside workshops

Building Alongside Software

Members learn software development while building complete robotic systems.

- Arduino and microcontroller programming
- C/C++ algorithms and Arduino sketches
- Sensor interfacing and electronics
- Project simulations and virtual testing
- AI integration into robotics projects

Code → Simulate → Build → Test

Project Demonstrations

Projects developed within the club will be showcased once a working prototype or significant milestone has been achieved.

- Demonstration of working systems
- Explanation of design and implementation
- Discussion of applications and future improvements

Your Ideas Matter

Yantrika is not limited to predefined club projects.

- New ideas from members are always welcome
- Independent projects can be developed and showcased
- Members are encouraged to propose and lead projects
- The club grows through collaboration and innovation

Build Our Autonomous Drone

- **C Programming** — Control logic and algorithms
- **Embedded Electronics** — Microcontrollers, ESCs and motors
- **Sensors** — IMU, LiDAR, GPS and cameras
- **Computer Vision** — Object detection and visual tracking
- **Autonomous Navigation** — Mapping, localization and path planning

Build Our First Autonomous Robot

- Mechanical system
- Embedded electronics
- Sensors
- Computer vision
- Autonomous navigation

Let's build it.

How to Join

- **Who can join?**

- Students of IACS
- Beginners and experienced students
- Anyone interested in robotics, electronics, coding, or research

- **What happens next?**

- 1 Register as a Yantrika member
- 2 Choose your areas of interest
- 3 Attend introductory workshops
- 4 Join a project team
- 5 Start building

**You don't have to know everything.
You just have to start.**

And that's the point.

Yantrika is not just a club
with projects to show you.

It is a place where we build them together.

Welcome to Yantrika.

Questions?

Welcome to Yantrika!

Imagine. Build. Experiment. Repeat.